ReAct

Reason + Act
Moving from reasoning to planning

NEED FOR PLANNING

Reactive (non-planning) agents optimise next action, whereas many tasks require optimising a sequence of actions

Planning introduces a global structure over local decisions, converting the agent from reactive to deliberative

Where reactive approaches fail:

- **Myopic decision-making**
  - Chooses locally optimal actions
  - Ignores long-term dependencies
  - Example: Query requires multi-hop retrieval, agent stops after first plausible answer
- **No task decomposition**
  - Complex tasks require subgoal identification and ordering constraints
  - Without planning, steps are missed and execution is incomplete
- **No recovery strategy**
  - When a step fails, reactive agents retry blindly
  - No structured backtracking
- **Tool misuse**
  - Without a plan, tools are selected opportunistically
  - This leads to redundant or irrelevant calls

ReAct: REASON AND ACT

ReAct planning interleaves reasoning traces with tool actions in a single loop

It is the same loop we built, but with one new rule. *Every action must be preceded by an explicit, visible thought.*

> **Without ReAct:**
>
> `Observe → [LLM reasons internally] → tool_use block → update → repeat`
>
> **With ReAct (Yao et al., 2022):**
>
> `Observe → Thought:[explicit reasoning] → Action → Observe:[tool result] → thought → …`

The agent writes its reasoning before it acts. The reasoning is a part of the output.

You can read the trace and see exactly why each tool was called.

USING REACT-STYLE PLANNING

We follow a strict structure that makes reasoning auditable at every step

To implement a ReAct-style agent,

```
loop:
    generate Thought
    if action needed:
        execute tool
        append Observation
    else:
        return answer
```

An example trace could look like:

```
Thought: I need current data → use search
Action: search("GDP of India 2025")
Observation: retrieved data

Thought: Now compute growth
Action: calculator(...)
Observation: result

Thought: I can answer now
Final Answer: ...
```

THOUGHTS IN REACT

Three types of thoughts are generated during ReAct-based planning

Planning thought

- Before first action

  'I need to:
  search for the release year,
  get current year,
  calculate.'

- Signal: agent is decomposing the task; good sign

Reasoning thought

- After an observation

  'I found that Django was released in 2005. Now I need Flask's date before I can compare.'

- Signal: agent is integrating results; normal behaviour

Completion thought

- Before final answer

  "I now have all three frameworks' ages. I can write the comparison."

- Signal: agent believes goal is met; Check whether true

- **Chain-of-Thought** prompting promotes internal reasoning only without environmental interaction
  Reason → Answer
- ReAct extends CoT to iterative reasoning and helps in grounded reasoning with dynamic tool use
  Reason ↔ Act ↔ Observe

WHERE REACT EXCELS

ReAct shines on reactive chains, tasks with multiple tool calls needing auditable traces, and reasoning serving as information synthesis

Customer operations: *Support triage and IT helpdesk*

- Support: *look up account → check subscription tier → query known issues → draft resolution*
- IT helpdesk: *parse error code → search runbook → check recent deployments → recommend fix*
- Each observation narrows the next action. The Thought trace doubles as a compliance audit log.

Sales and research intelligence: *Prospecting and competitive analysis*

- Sales: *find company's latest funding round → identify decision-maker → retrieve recent news → draft personalised outreach*
- Competitive: *pull rival pricing → calculate delta → flag outliers → summarise*
- Multi-hop lookup where each result determines what to retrieve next.

Engineering and ops tooling: *Debugging and incident response*

- CI/CD: *read failed build log → look up error → check recent commits → propose fix*
- Incident: *receive alert → query metrics → look up runbook → propose remediation*
- Strict sequential dependency, and an auditable trace is needed here.

It has known limits on complex tasks

REACT LIMITATIONS

Lookahead *Structural*
Task requires a strategy before the first action. ReAct commits to step 1 without knowing what step 5 will need.

Goal decomposition *Structural*
Complex goals are trees, not chains. ReAct serialises everything and can run out of steps before full coverage.

Dead end recovery *Operational*
When a tool fails, ReAct has no backtracking mechanism. It can only retry the same path or give up.

Cost, latency, and context erosion *Operational*
Each iteration is a separate LLM call over the full history. Token use is ~5–10× higher than streamlined patterns.

Reliability *Operational*
Infinite loops, format brittleness, and significant variation in behaviour across different models and parallism limits.

- So, the ReAct method is generally classified as a **dynamic, on-the-fly reasoning strategy** rather than a traditional "agentic planning" strategy
- While it involves "planning" in the sense that the agent decides its next move, it is distinct from architectures that use a formal planning phase

REACT vs PLANNING

Requires	ReAct	Needs Planning
Sequential tool calls	✓	
Multi-hop retrieval	✓	
Interleaved reasoning and acting	✓	
Strategy before first action		✓
Multi-branch task structure		✓
Backtracking from dead ends		✓
Parallel subgoal execution		✓

The rule of thumb: use ReAct when the task is a chain. Use planning when the task is a tree.

Task Decomposition

How planning agents break complex goals into focused, executable subtasks

THE PROBLEM WITH FLAT AGENTS

A single-loop agent picks the most immediately obvious tools and often misses other workstreams entirely

Coverage drift
With 6 tools and a complex multi-part task, the agent tends to exhaust 2-3 tools on one workstream and never reaches the others.

No structure
The agent has no internal representation of which aspects of the task it has and hasn't addressed.

Opportunistic selection
Each tool call is independently chosen; there is no mechanism to ensure balanced, comprehensive coverage.

Compounding bias
Early tool results reinforce exploration of one thread, making it even less likely the agent revisits other workstreams.

TASK DECOMPOSITION

Task decomposition is the process of transforming a complex objective into a structured set of smaller, solvable subgoals

Subgoal generation is the act of deriving those units such that their completion implies solving the original task

Given goal G:

$$\text{Decomposition: } G \rightarrow \{g_1, g_2, \ldots, g_n\}$$

Subject to:

- Completeness: solving all g_i solves G
- Ordering constraints: some g_i must precede others
- Minimality: avoid redundant subgoals

Types of decomposition:

- **Sequential** (linear plan): `g₁ → g₂ → g₃`
- **Parallel** (independent subgoals): `g₁, g₂, g₃ → aggregate`
- **Hierarchical** (recursive decomposition):

```
G
├── g₁
├── g₂ → {g₂₁, g₂₂}
```

- **Conditional:** `If condition → g₁; Else → g₂`

SUBGOAL GENERATION

A well-formed subgoal should be:

- **Atomic (or near-atomic):** Executable in a single reasoning/action step
- **Verifiable:** Has a clear success condition
- **Independent (where possible):** Minimises coupling with other subgoals
- **Ordered when necessary:** Explicit dependencies like $g_1 \rightarrow g_2 \rightarrow g_3$

To decompose the task into quality subgoals, we can use several subgoal generation strategies:

- **Direct prompting:** Ask model: “Break this task into steps”
- **Structured planning prompts:** Force step-wise format
- **Tool-aware decomposition:** Generate subgoals aligned with available tools, like retrieval, computation, and synthesis steps
- **Iterative refinement:** Generate coarse plan and refine each step

COMPONENTS

Planning agents solve coverage drift by decomposing the goal before executing

- **Decomposer (Phase 1)**
  - A dedicated LLM call receives the high-level goal and outputs a structured plan: a list of subtasks, each with its own goal and assigned tools
- **Executor (Phase 2)**
  - Each subtask runs its own focused mini-loop with only the tools assigned to it
  - The executor cannot drift into other workstreams
- **Plan Manager**
  - Orchestrates the two phases
  - Detects failures, triggers replanning when early findings change what the plan needs to cover
- **Synthesis**
  - After all subtasks complete, a final LLM call combines subtask findings into a coherent answer

THE DECOMPOSER

What it does

- Receives the full task + list of available tools
- Outputs a JSON plan: list of subtasks with `id`, `goal`, `tools[]`, `priority`
- Each subtask gets only the most relevant 1-3 tools
- Priority ordering ensures dependencies are resolved first

Why it matters

- Tool scoping: each executor only sees its own workstream's tools
- Priority ordering: subtask 1 results can inform subtask 3 execution
- Separation of concerns: planning logic is isolated from execution logic
- The plan is inspectable and modifiable before execution begins

THE EXECUTOR

Mini agent loop per subtask

- Receives only the subtask goal (not the full original task)
- Has access only to the tools assigned in the plan
- Runs for at most N steps (step budget)
- Forces a summary if budget is exhausted
- Returns structured findings for synthesis

Step budget importance

- Without a budget, one ambitious subtask can monopolise the context window
- 3 steps is sufficient for most single-workstream subtasks
- Budget exhaustion triggers a forced summary, not an abort
- Predictable cost:
  *N subtasks x M steps = N*M* tool calls maximum

DYNAMIC REPLANNING

The plan is not fixed at decomposition time. The plan manager updates it as evidence arrives.

Replanning triggers

- After each subtask, check findings for trigger conditions (e.g., a prerequisite dependency discovered)
- If triggered, call the decomposer again with the new context

Recovery subtasks

- When a subtask fails (empty result, tool error), the decomposer is called with the failure context and produces 1-2 recovery subtasks targeting different tools

Structural vs prompt control

- Replanning logic must be engineered into the plan manager
- Prompting the executor to 'adapt if needed' does not work, the executor has no visibility of the broader plan

Synthesis is always the last step

- Regardless of how many replanning rounds occur, synthesis happens once, after all subtasks complete
- This prevents early conclusions from biasing later tool calls

Tree-Search Planning

When the optimal sequence of subtasks is not known in advance

WHEN LINEAR DECOMPOSITION IS NOT ENOUGH

Linear decomposition works when all subtasks can be identified upfront

Unknown dependencies

When the output of one subtask determines which subtask should run next, a fixed plan cannot capture all the branching possibilities.

Uncertain tool reliability

When some tools may fail or return insufficient information, the agent needs a fallback path, and the best fallback may depend on what the primary path returned.

Optimisation over sequences

Some tasks have many possible orderings of subtasks, and the best ordering depends on intermediate results. A tree representation makes this explicit.

Deep compositional reasoning

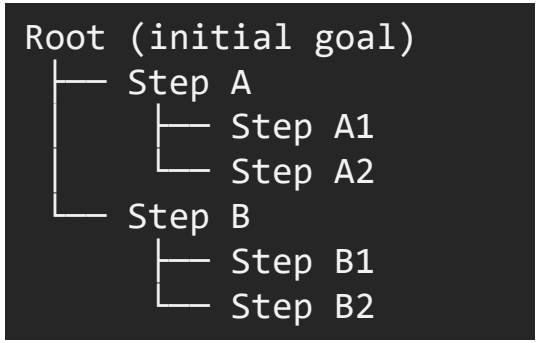
Multi-hop research, code planning, or structured argument analysis may require exploring branches before committing to a path.

PLANNING AS TREE SEARCH

Planning can be formalised as search over a space of partial solutions

- State: current context (goal + intermediate results)
- Action: a candidate next step / subgoal
- Transition: applying an action updates the state
- Goal test: solution satisfies the task

This induces a **search tree**, where each path is a candidate plan:



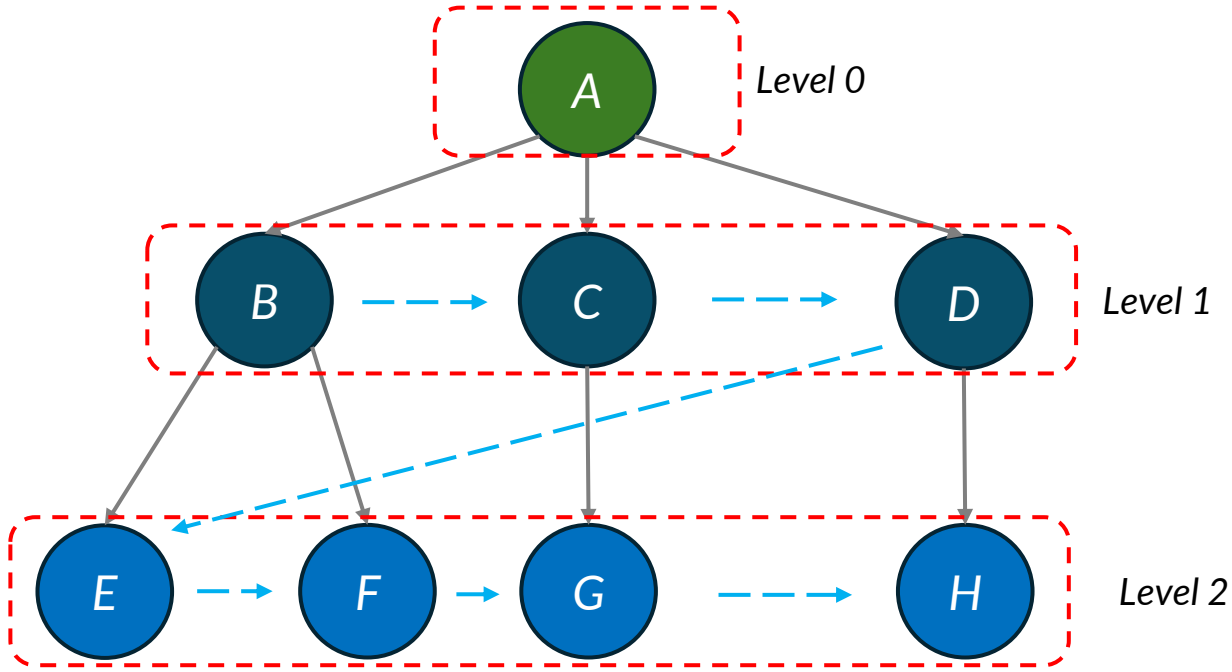
Several strategies exist to search over these trees

BREADTH-FIRST SEARCH

Generate multiple alternative first steps and expand all possibilities evenly

Breadth-First Search (BFS)

- Explores all subtasks at a depth before going deeper
- Guarantees finding the shallowest valid plan
- High cost: evaluates many branches before committing
- Best when: shallow plans exist, cost per node is low

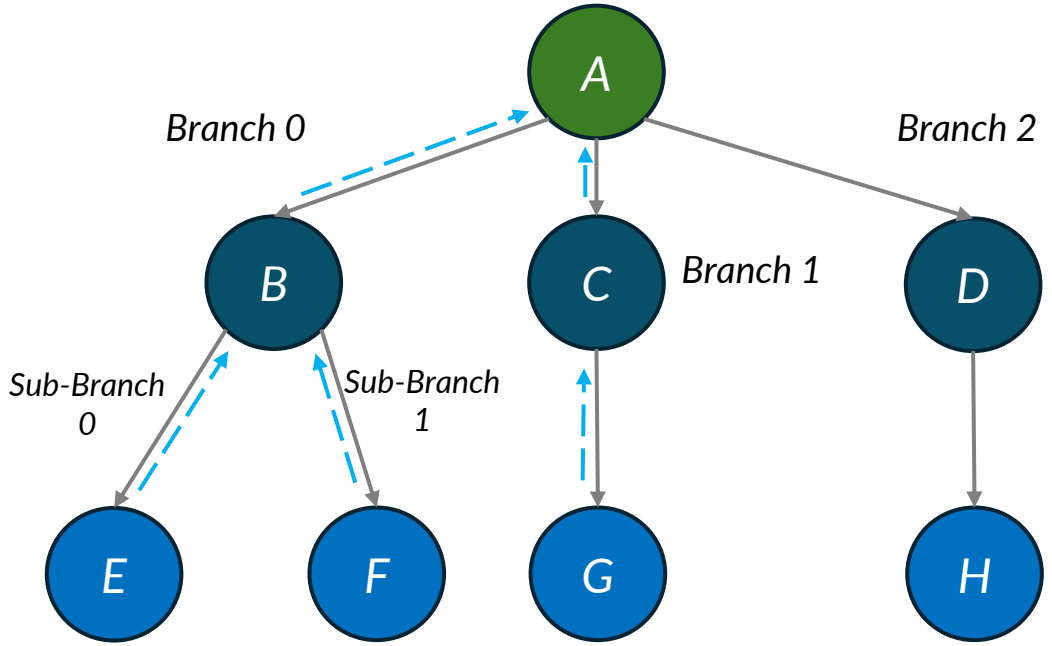


DEPTH-FIRST SEARCH

Commit to a plan early
Execute step-by-step until success/failure

Depth-First Search (DFS)

- Commits to one branch and goes deep before backtracking
- Low memory cost; may find a plan quickly
- Risk: goes deep on a poor branch before backtracking
- Best when: search space is narrow, one path is likely correct

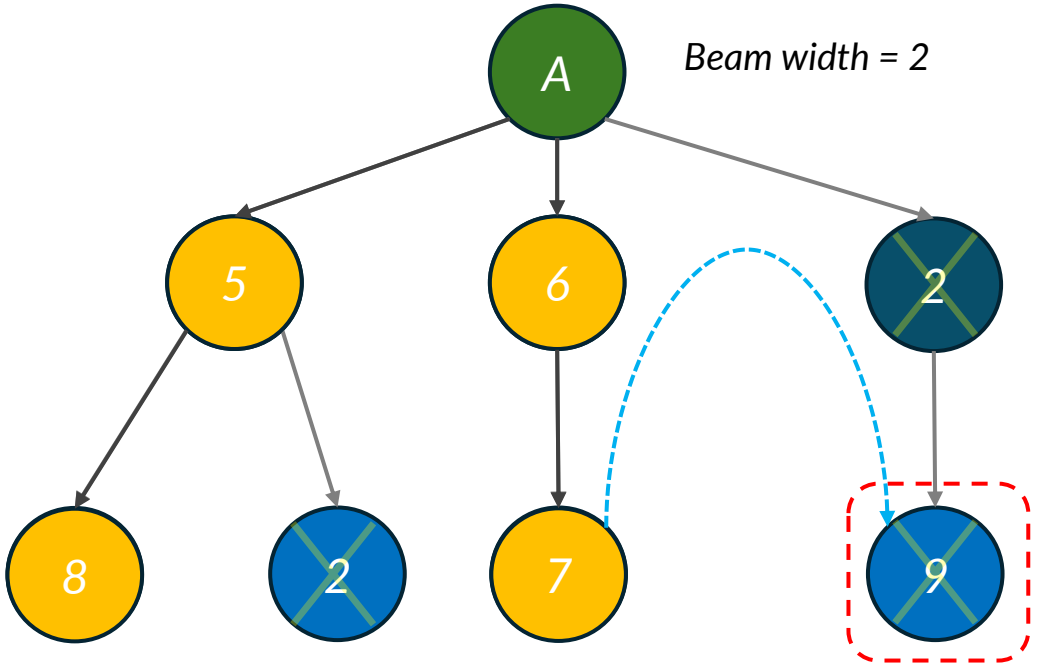


BEAM SEARCH

Generate candidate steps, score them (via LLM or heuristic)
and expand highest-scoring path

Best-First / Beam Search

- Maintains a scored frontier; expands highest-score nodes first
- Score = estimated likelihood of reaching the goal
- Beam width controls exploration vs exploitation tradeoff
- Best when: a heuristic quality signal is available per node



PLANNING DEPTH vs COST

Deeper planning finds better plans...

- More tool calls evaluated before committing
- Higher probability of finding the optimal subtask sequence
- Catches dead-end branches before execution
- Better recovery path selection when tools fail

...but increases cost significantly

- Every node in the tree requires at least one LLM call to evaluate
- Exponential growth: branching factor b at depth $d = b^d$ nodes
- Token cost per node includes full context up to that node
- Latency compounds: user waits while agent explores the tree

LAZY PLANNING

- Lazy planning avoids the full tree cost by expanding only when necessary
- In lazy planning,
  - do not generate the full plan upfront
  - generate only the next subtask, execute it
  - then decide whether to continue, branch, or stop based on what was found
- **Use it when**
  - the search space is large
  - most branches are prunable early
  - the correct next step is usually clear from the previous result
- **Tradeoff vs eager planning**
  - Lazy planning has lower upfront cost but cannot backtrack
  - Eager planning (full tree) is expensive but can choose the globally best path
- **Hybrid approaches**
  - Generate the first 2-3 subtasks eagerly (plan the immediate known work), then switch to lazy planning for deeper steps where information is needed to decide

CHOOSING A PLANNING STRATEGY

Match the planning strategy to the task structure

Strategy	Use when	Main cost
Linear decomposition	All subtasks known upfront	Low
BFS tree search	Shallow plan guaranteed	High (breadth)
DFS tree search	One path likely correct	Medium (depth)
Best-first search	Quality signal available	Medium + heuristic
Lazy planning	Most branches prunable	Low per step
Hybrid (Eager + Lazy)	First 2-3 steps are known	Low to medium

Planning Failure Modes

Three structural failure modes that arise specifically from the planning layer

RIGID PLANNING

What happens

- Plan is generated once and executed without revision
- Mid-execution findings that may contradict the plan are ignored
- New evidence is ignored and stale plan runs to completion
- No mechanism exists to add or modify subtasks after decomposition
- Subtasks that are irrelevant are still executed, wasting budget

Example & Fix

Regulatory subtask finds BSI C5 needs ISO 27001 (9-15 months prerequisite)

Rigid agent continues to financial modelling without flagging the blocker

- Fix: replanning triggers scan each subtask result for trigger keywords
- On trigger: decomposer called again with failure context to add recovery subtasks

OVER-DECOMPOSITION

What happens

- Decomposer prompt says 'be thorough' with no subtask count limit
- LLM interprets thoroughness as more subtasks
- Context is exhausted before synthesis
- Many micro-subtasks generated, each covering one narrow data point
- Later subtasks never execute and token budget gets exhausted

Example & Fix

Final report looks thorough (long plan) but is missing workstreams

Later subtasks are simply never run; synthesis has incomplete inputs

- Fix: `max_subtasks = 4` enforced in the decomposer prompt
- Decomposer must consolidate related concerns into fewer broader subtasks

*Similarly, you can also face **under-decomposition**, when the steps are still too complex and the executor fails to carry them out*

NO STEP BUDGET

What happens

- One ambitious subtask has all tools available and no step limit
- The executor calls 6-8 tools, covering multiple workstreams
- This exhausts most of the context window and token budget
- Remaining subtasks receive truncated histories or fail entirely

Example & Fix

Final report is comprehensive on one workstream, thin on others

- Root cause: one subtask monopolised the execution budget
- Fix: `step_budget=3` per subtask enforced in the executor
- A constrained answer is better than starving all subsequent subtasks

MITIGATIONS: STRUCTURAL CONTROLS

Replanning Triggers

- Keyword scan after each subtask
- Fires when critical new evidence contradicts the plan
- Calls decomposer with failure context to add recovery subtasks
- Cannot be replaced by prompting the executor

Subtask Count Cap

- `max_subtasks=N` enforced in decomposer prompt
- Forces consolidation of related concerns
- Protects context budget for all subtasks + synthesis
- Typical cap: 4-5 for a 4-workstream task

Per-Subtask Step Budget

- `step_budget=3` enforced in the executor
- Triggers a forced summary on budget exhaustion
- Guarantees predictable, bounded cost per subtask
- 3 steps: *primary call + one follow-up + one clarification*

A good strategy is to **monitor the agent's trace**, test **perturbations in the output** from small task modifications, and keep **checking the tool alignment**